Loop Gestión

Manual Técnico

Sebastián Grande Moreno y Juan Carlos Díaz Cifuentes

Docente: MGTR. Ing. Jairo Armando Salcedo Aranda

Universidad de San Buenaventura, Bogotá · Optativa II (10348) — Desarrollo de Aplicaciones para Dispositivos Móviles · Noviembre de 2026

Tabla de contenido

Información del documento 3

1. Arquitectura 4

2. Estructura del repositorio 5

3. App Android 6

4. API Node.js / Express 7

5. Base de datos 8

6. Integraciones 11

7. Pruebas y CI 12

8. Mantenimiento 13

Información del documento

Campo	Valor
Proyecto	Loop Gestión — Plataforma académica full-stack con automatización e IA
Documento	Manual Técnico
Estándar de referencia	IEEE 1016 / IEEE 26514
Versión	1.0
Autores	Sebastián Grande Moreno y Juan Carlos Díaz Cifuentes
Asignatura	Optativa II (10348) — Desarrollo de Aplicaciones para Dispositivos Móviles
Docente	MGTR. Ing. Jairo Armando Salcedo Aranda
Institución	Universidad de San Buenaventura, Bogotá
Fecha	Noviembre de 2026

> **Nota sobre el alcance de este documento.** Se elaboró a partir del proyecto en Azure DevOps (backlog, sprints, tablero, repositorio y pipeline, corte del 4 de noviembre de 2026) y de los mockups de la aplicación. El código fuente se encuentra en el repositorio de Azure Repos del equipo. Los datos que dependen directamente del código (nombres exactos de rutas, tablas, variables y versiones) están marcados con ⚠️ **Por confirmar con el código** para validarlos antes de la entrega final.

1. Arquitectura

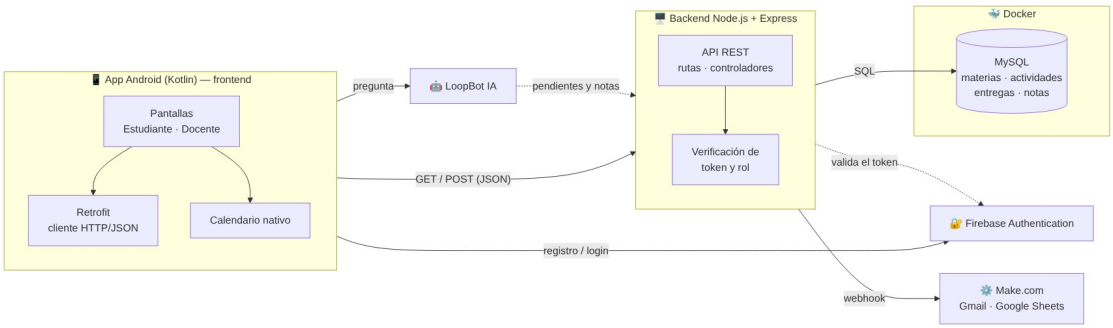


Figura 1. Arquitectura

Componente	Tecnología	Uso
App móvil (frontend)	Android nativo en Kotlin	Pantallas de estudiante y docente
Comunicación con la API	Retrofit (HTTP/JSON)	Peticiones GET/POST a la API REST
API (backend)	Node.js + Express	Lógica de negocio y endpoints REST
Base de datos	MySQL en contenedor Docker	Materias, actividades, entregas y notas
Autenticación	Firebase Authentication	Registro e inicio de sesión por rol
Automatización	Make.com (Webhook, Google Sheets, Gmail)	Notificaciones por correo y registro en hojas
Asistente	LoopBot IA	Responde dudas sobre pendientes, notas y temas
Calendario	Calendario nativo de Android	Agregar actividades al calendario del celular
Integración continua	Azure Pipelines (YAML, Node.js)	Ejecución automática del pipeline
Gestión del proyecto	Azure Boards (Scrum)	Backlog, sprints, tablero y dashboard

2. Estructura del repositorio

Según el repositorio de Azure Repos (Loop-Gestion):

```
Loop-Gestion/
├── docs/              documentación del proyecto
├── src/               código fuente (app Android y API Node.js)   ⚠ Por confirmar con el código: subcarpetas exactas
├── tests/             pruebas automatizadas (incluye chatbot.test.js)
├── azure-pipelines.yml   pipeline de integración continua
└── README.md          descripción del proyecto
```

3. App Android

- **Lenguaje:** Kotlin. **Red:** Retrofit con convertidor JSON.
- **Patrón recomendado:** pantalla → ViewModel → repositorio → servicio Retrofit (⚠️ Por confirmar con el código: arquitectura exacta usada).
- **Autenticación:** Firebase Authentication; la app obtiene el *ID token* del usuario y lo envía a la API en el encabezado `Authorization: Bearer <token>`.
- **Calendario:** se usa un *Intent* de Android para crear el evento en el calendario del teléfono.
- **URL de la API:** en el emulador, `http://10.0.2.2:<puerto>` apunta al computador; en un celular físico, la IP del computador o el dominio publicado.

4. API Node.js / Express

- Rutas agrupadas por recurso (usuarios, materias, actividades, entregas, notas, chatbot).
- Un *middleware* verifica el token de Firebase y el rol antes de cada ruta protegida.
- Conexión a MySQL con un *pool* de conexiones; credenciales tomadas de variables de entorno (.env).
- Al crear una actividad, la API envía un POST al webhook de Make.com.

Método	Ruta (referencia)	Uso	Rol
GET	/api/materias	Materias del usuario	Todos
GET	/api/actividades?materia={{id}}	Actividades de una materia	Todos
POST	/api/actividades	Crear actividad	Docente
POST	/api/entregas	Enviar entrega	Estudiante
PUT	/api/entregas/{{id}}/calificacion	Calificar	Docente
GET	/api/notas	Boletín	Estudiante
POST	/api/chatbot	Pregunta a LoopBot	Estudiante

5. Base de datos

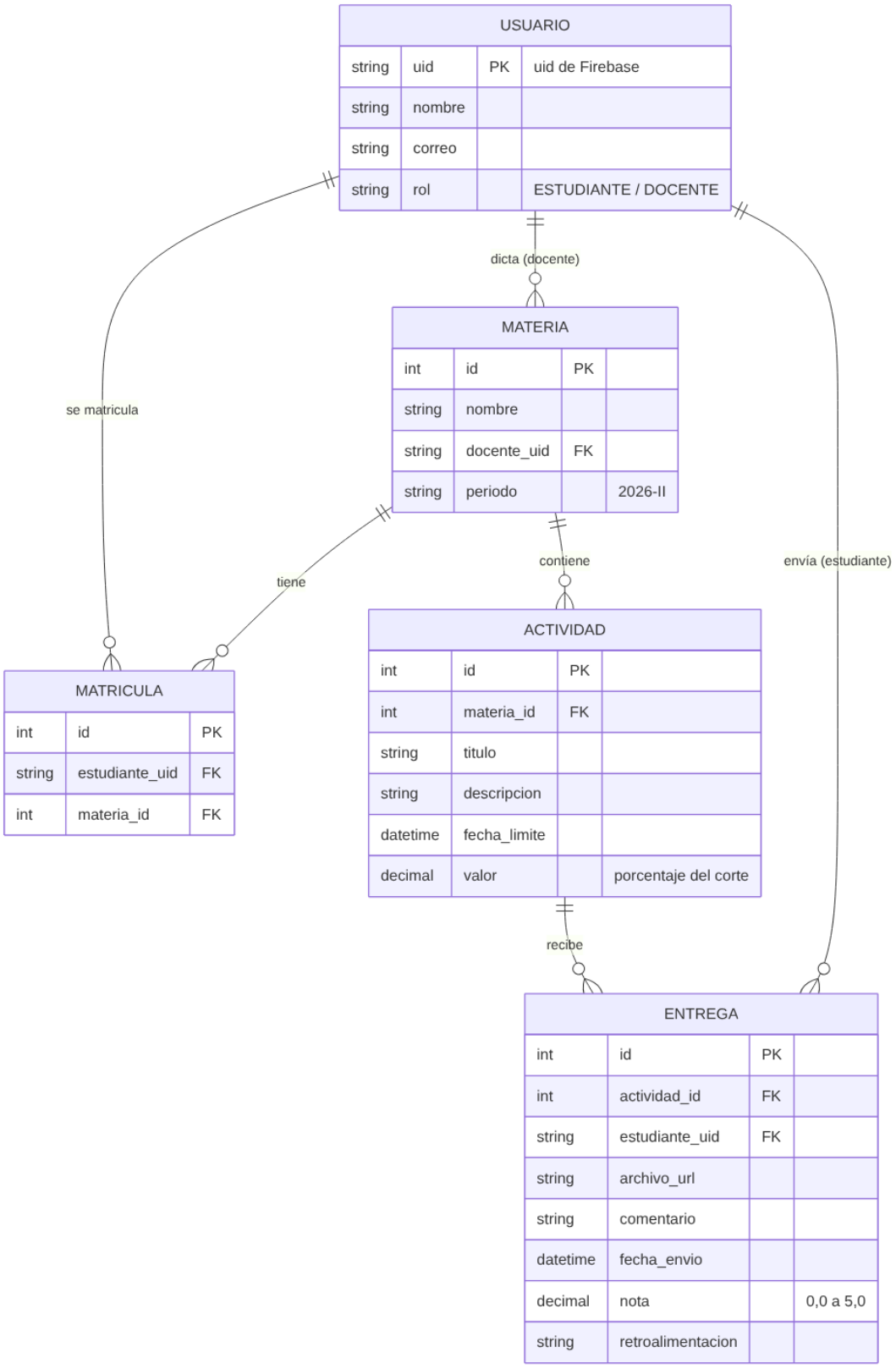


Figura 2. Modelo entidad-relación conceptual

Entidad	Descripción
USUARIO	Estudiantes y docentes; se enlaza con Firebase por el uid.
MATERIA	Asignatura del periodo, con su docente.
MATRICULA	Relación estudiante–materia.
ACTIVIDAD	Tarea con fecha límite y valor porcentual.
ENTREGA	Archivo o enlace del estudiante, con nota y retroalimentación.

6. Integraciones

Servicio	Cómo se integra	Datos que recibe
Firebase Authentication	SDK en la app; verificación del token en la API	Correo y contraseña (los guarda Google)
Make.com	Webhook HTTPS llamado por la API	Título, materia, fecha y lista de correos
Gmail (vía Make)	Módulo de Make.com	Correo a los estudiantes
Google Sheets (vía Make)	Módulo de Make.com	Una fila por actividad publicada
LoopBot IA	Endpoint de la API que consulta la IA ⚠️ Por confirmar con el código	Pregunta del estudiante y su contexto académico

7. Pruebas y CI

- Pruebas automatizadas en tests/ (incluye chatbot.test.js); se ejecutan con npm test (⚠️ Por confirmar con el código: framework, por ejemplo Jest).
- Pipeline azure-pipelines.yml: instala Node.js y ejecuta los pasos del proyecto; corrida #20261104.1 en verde. Reemplazar NodeTool@0 por UseNode@1.

8. Mantenimiento

Tarea	Cómo hacerlo
Agregar un endpoint	Nueva ruta + controlador en la API; proteger con el middleware de rol; agregar el método en la interfaz Retrofit.
Cambiar la estructura de la BD	Script SQL de migración versionado; reconstruir el contenedor.
Cambiar el correo de notificación	Editar el escenario en Make.com; no requiere cambiar código.
Respaldar la base de datos	docker exec \<contenedor> mysqldump -u \<usuario> -p \<bd> > respaldo.sql